← Back to Unit 2 Index

QUESTION 1 • UNIT 2 – PERCEPTRONS

What is a Perceptron? What are the steps involved for training a Perceptron in Deep Learning?

📝 5 Marks

The Perceptron & Its Training Process

Complete answer with diagrams and step-by-step explanation

What is a Perceptron?

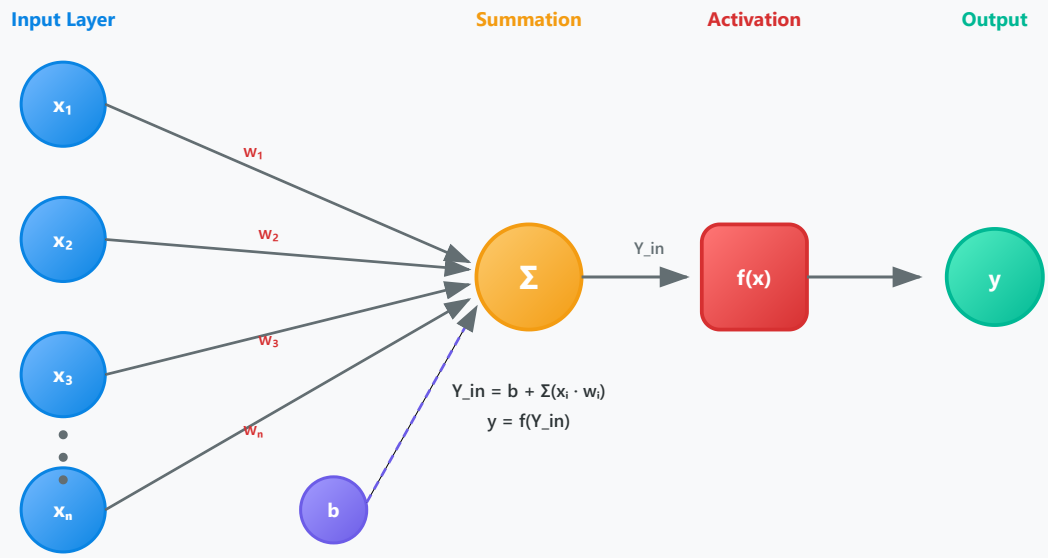
A Perceptron is the simplest type of artificial neural network, consisting of a single-layer feed-forward network. It was introduced by **Frank Rosenblatt** in 1957 and serves as the fundamental building block of modern deep learning.

It has an **input layer** and an **output layer**, where the input layer receives signals and the output layer processes them to produce results. The perceptron uses weights on synaptic links to connect input neurons to output neurons, and the output is computed using an activation function.

The perceptron network consists of three types of units:

- **Sensory (Input) Units** — receive external input signals
- **Associate (Hidden) Units** — process intermediate computations
- **Response (Output) Units** — produce the final classification result

Figure 1 — Architecture of a Single-Layer Perceptron



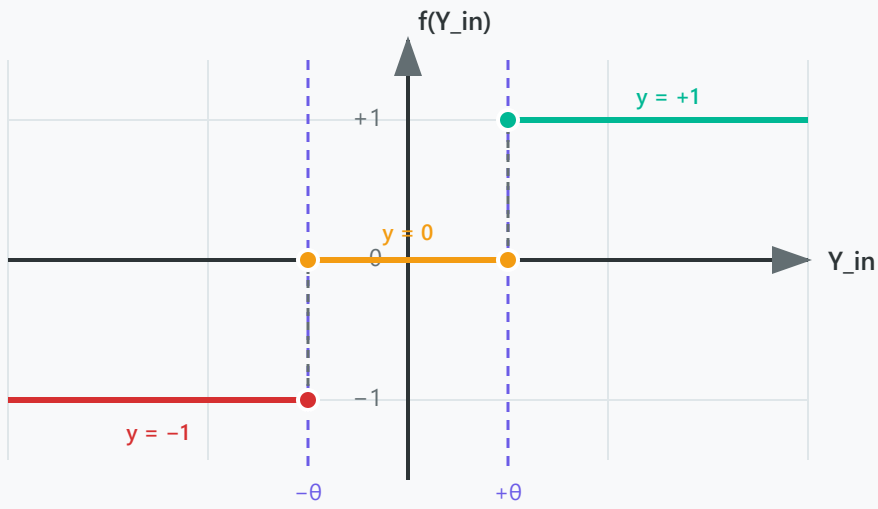
A perceptron receives inputs $x_1 \ldots x_n$, multiplies each by its weight, sums with bias b, and passes through an activation function f to produce output y.

Binary Step Activation Function

The perceptron uses a **binary step activation function** with a fixed threshold (θ). The function categorizes the net input into one of two classes:

$$f(Y_{in}) = \begin{cases} +1 & \text{if } Y_{in} > \theta \\ 0 & \text{if } -\theta \le Y_{in} \le \theta \\ -1 & \text{if } Y_{in} < -\theta \end{cases}$$

Figure 2 — Binary Step Activation Function



The binary step activation function outputs +1, 0, or −1 depending on whether the net input exceeds the threshold θ.

Steps for Training a Perceptron

The perceptron follows an iterative supervised learning algorithm to adjust its weights until the output matches the desired target. Below are the detailed steps:

1 Initialization

Set initial **weights** (w_1, w_2, ..., w_n) and **bias** (b) to *zero* or small random values. Define the learning rate η (usually between 0 and 1).

2 Input Assignment

Assign input values (x_1, x_2, ..., x_n) to the input layer neurons for each training sample.

3 Net Input Calculation

Compute the **net input** (weighted sum) using the formula:

$$Y_{in} = b + \Sigma_i (x_i \cdot w_i)$$

where x_i are input values, w_i are corresponding weights, and b is the bias.

4 Apply Activation Function

Use the **activation function** (binary step function) to calculate the output:

$$y = f(Y_{in})$$

where f is the binary step activation function with threshold θ.

5 Compare Output

Compare the calculated output **y** with the desired **target output t**. If $y \neq t$, there is an error that must be corrected.

6 Weight & Bias Adjustment

If there is an error ($y \neq t$), update the weights and bias using the Perceptron Learning Rule:

$$w_i^{(new)} = w_i^{(old)} + \eta \cdot x_i \cdot t$$

$$b^{(new)} = b^{(old)} + \eta \cdot t$$

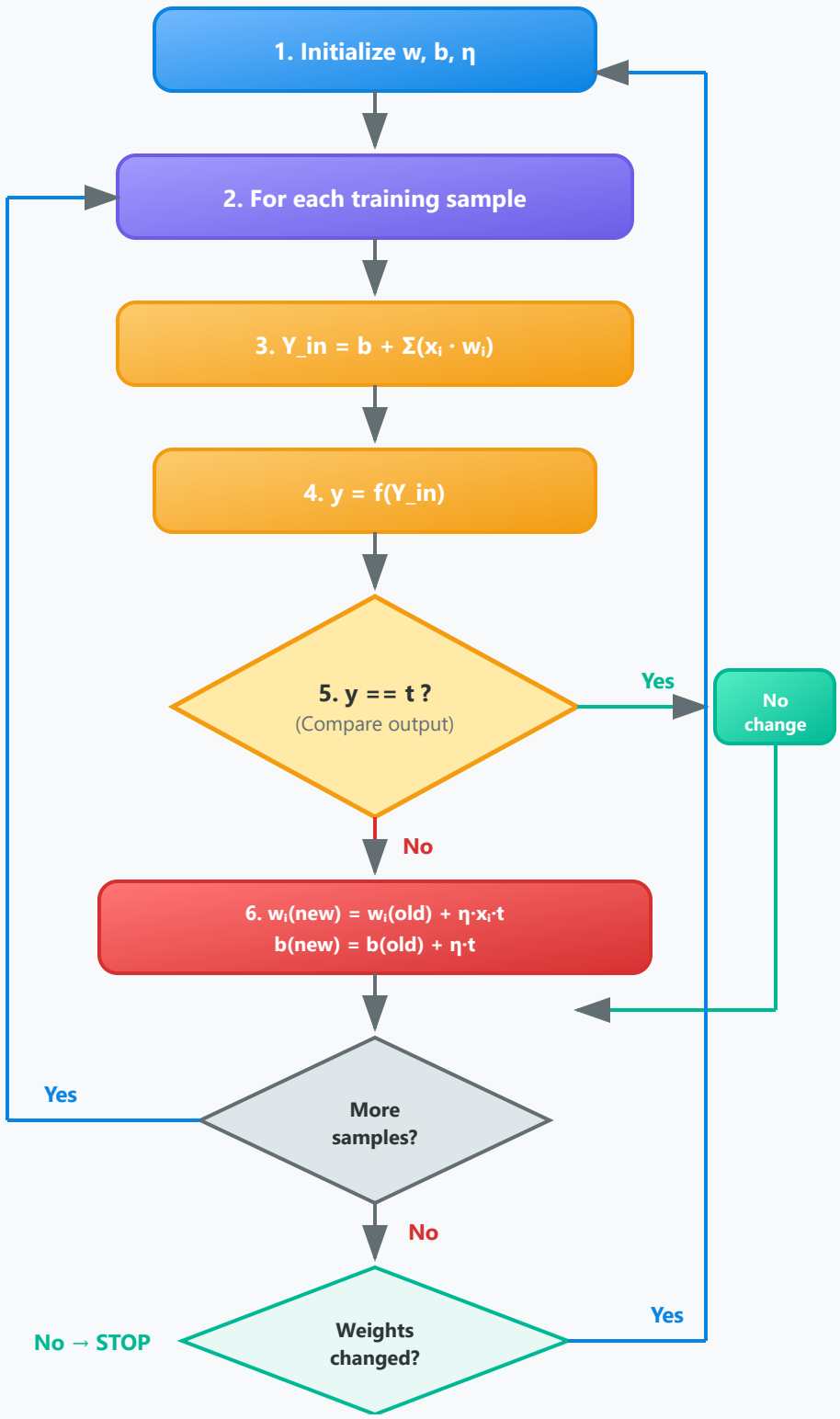
Where η is the learning rate, t is the target value (+1 or −1), and x_i is the input value.

7 Repeat (Convergence)

Repeat steps 3–6 for **all training patterns** until the stopping condition is met — i.e., no change in weights or the error is minimized across all samples.

Training Flowchart

Figure 3 — Perceptron Training Algorithm Flowchart



The training loop continues until no weight changes occur across an entire epoch — the perceptron has converged.

Key Points

- The perceptron network consists of **sensory (input)**, **associate (hidden)**, and **response (output)** units.
- It uses a **binary step activation function** with a fixed threshold.
- The **perceptron learning rule** adjusts weights based on errors in the output.
- The perceptron can only learn **linearly separable** patterns (e.g., AND, OR but not XOR).
- It forms the **foundation** for more complex multi-layer neural networks used in deep learning.

Summary Table

Component	Symbol	Description
Inputs	$x_1, x_2, \ldots, x_n$	Feature values fed into the perceptron
Weights	$w_1, w_2, \ldots, w_n$	Learnable parameters multiplied with inputs
Bias	b	Additional learnable parameter added to weighted sum
Net Input	Y_in	$b + \Sigma(x_i \cdot w_i)$ — weighted sum of inputs plus bias
Activation	f(Y_in)	Binary step function with threshold θ
Output	y	Predicted class label (+1, 0, or −1)
Learning Rate	η	Controls how much weights change per update (0 to 1)

Component	Symbol	Description
Target	t	Desired output for the given input